Answering Constraint Path Queries over Graphs

Heyang Li
University of Kaiserslautern-Landau
Kaiserslautern, Germany
heyang.li@cs.rptu.de

Anthony Widjaja Lin
University of Kaiserslautern-Landau
and MPI-SWS
Kaiserslautern, Germany
awlin@mpi-sws.org

Domagoj Vrgoč
PUC Chile and IMFD Chile
Santiago, Chile
vrdomagoj@uc.cl

ABSTRACT

Constraints are powerful declarative constructs that allow users to conveniently restrict variable values that potentially range over an infinite domain. In this paper, we propose a constraint path query language over property graphs, which extends Regular Path Queries (RPQs) with SMT constraints on data attributes in the form of equality constraints and Linear Real Arithmetic (LRA) constraints. We provide efficient algorithms for evaluating such path queries over property graphs, which exploits optimization of macro-states (among others, using theory-specific techniques). In particular, we demonstrate how such an algorithm may effectively utilize highly optimized SMT solvers for resolving such constraints over paths. We implement our algorithm in MillenniumDB, an open-source graph engine supporting property graph queries and GQL. Our extensive empirical evaluation in a real-world setting demonstrates the viability of our approach.

CCS CONCEPTS

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

KEYWORDS

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:
Heyang Li, Anthony Widjaja Lin, and Domagoj Vrgoč. 2026. Answering Constraint Path Queries over Graphs. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 16 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Graph databases have been an increasingly popular technology in the database ecosystem over the past decades, with multiple open-source [20, 33, 37] and proprietary systems [29, 34–36, 38] being developed, and a steady stream of research literature on the subject [1, 2, 9]. Graphs offer an intuitive modeling of the application domain with nodes representing entities and edges representing connections between these entities. Edges are usually labeled to identify the type of the connection, giving rise to the edge-labeled graph database model [2, 6]. Extending this model with the ability to add attributes with their associated values to both nodes and edges (and also to label the nodes) is supported in the property-graph data model [1], which is now the industry standard.

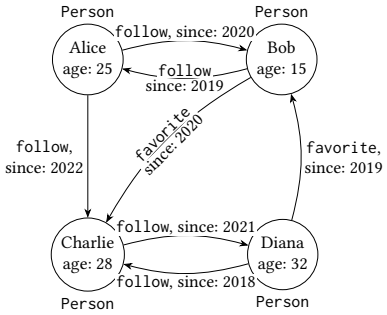


Figure 1: Social network property graph.

A core querying primitive over graph databases are *path queries*, which allow traversing the graph with paths whose length is not know in advance. Traditionally [1, 6, 17], the main class of such queries were *regular path queries (RPQs)*, which were specified via a regular expression and would return pairs of nodes connected by a path whose edge labels spell a word in the language of the expression. For example, in the social network graph in Figure 1 one might be interested in the people a certain person follows, expressed by the regular pattern $\texttt{follows}^+$, which traverses a series of `follows`-labeled edges. However, in practice one is also interested in property values occurring along these paths [9]. For example, in our previous query we might also want to check that all the links are rather new (e.g. established after 2020), or that the maximal age difference between people along this path is less than ten years. In our example the path `Alice`→`Charlie`→`Diana` is one such path connecting `Alice` to `Diana`.

While such queries, which mix path traversal with data constraints, are supported in languages like Cypher [14] or GQL [18] by collecting the relevant values in a list and post-process this list to check the constraints, as shown in [15], this approach can lead to inefficient query evaluation. Overall, this class of queries are known to be difficult to evaluate theoretically [12, 23] and finding the correct theoretical framework which can lead to implementable algorithms for path queries with data constraints has been identified as a relevant open problem in a recent survey of the GQL query standard [9]. In this paper we propose *parametric regular path queries* as a potential answer to this question and show that

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06…$15.00
https://doi.org/XXXXXXX.XXXXXXX

this class of queries: (i) has good theoretical properties; (ii) is easily implementable on top of existing path processing frameworks; and (iii) allows expressing queries mixing graph traversal with data constraints.

Existing approaches. Apart from collecting relevant nodes and edges along a path into a list that is to be validated upon finishing the traversal, which seems to be a common approach in existing systems (seen [15]), there were efforts in the theory community to provide a characterization of such queries using expressions and automata with registers [12, 23]. In essence, registers allow storing data encountered along a path into a finite set of registers (or variables) and compare their content [23], or additionally allow arithmetic combinations of such parameters to be checked against register data [12]. These approaches are somewhat heavy-handed, as the complexity bounds they derive are rather high, and they were never converted to practical algorithms.

Contributions. To the best of our knowledge, this is the first paper to make substantial contributions to *both theory and practice* of evaluating path queries with data constraints. For this, we use parameters to specify complex data constraints that can be checked while traversing arbitrary length paths. This results in a lightweight theoretical framework for handling such tests, as opposed to previous work which used registers [23] or registers in conjunction with parameters [12], both of which exhibit much higher evaluation complexity. In particular, in this work we show:

- That the use of parameters significantly reduces evaluation complexity in comparison with approaches that leverage registers [12, 23], while at the same time preserving high expressive power and allowing a close connection with automata model.
- That our approach extends itself to efficient evaluation algorithms. In particular, we leverage the connection with automata to show that the framework for path finding proposed in [10, 26] can be extended to support parameters and result in algorithms which are easily implementable by extending the existing solutions.
- That our approach is implementable and that is scales. For this, we implement parametric regular path queries in MillenniumDB [37], an open-source graph engine supporting property graph queries and GQL, and do an extensive experimental evaluation over a series of real world graph datasets.

Related work. Apart from using registers [12, 23] to handle paths with data constraints in the graph setting, there is also a rich body of literature on *constraint queries* over databases in general [21], not only over graph databases. In this setting, also known as constraint-databases, one allows variables over a possibly infinite domain (e.g. the set of real numbers), which could be constrained by using formulas over certain logical theories (e.g. Linear Real Arithmetic (LRA)). Constraints were studied systematically in database theory in the 1990s resulting in several prototypes including DEDALE [16], MLPQ [30], and DISCO [5], although such systems were limited to databases not exceeding hundreds of tuples. To the best of our knowledge, none of these systems are still maintained and available in the public domain. Recently, constraints have been revisited in the context of graph databases [12, 13, 24]. In particular, a large class of constraint path queries can be answered efficiently (i.e. in nondeterministic logarithmic space and polynomial time, for any fixed query). Unfortunately, all of these algorithms rely on heavy machinery from embedded finite model theory called *Restricted Quantifier Collapse (RQC)*. In fact, the proposed algorithms rewrite a given constraint query into a simpler one that is in the worst case *doubly exponentially larger* than the original query!

Organization. The remainder of the paper is organized as follows. Preliminary definitions are given in Section 2. We define parametric RPQs and their automaton model in Section 3. Algorithms for their evaluation are given in Section 4 and Section 5. Experimental evaluation is in Section 6. We conclude in Section 7.

2 PRELIMINARIES

Here we define basic notions used throughout the paper.

Graphs and paths. Following the usual conventions in the research literature [8], we define *property graphs* as directed graphs where both edges and nodes carry labels and a series of attributes (i.e. properties) with their associated values. Formally, we assume disjoint countably infinite sets Vertices of vertex identifiers, Edges of edge identifiers, and Labels of edge labels. Similarly, we assume a countably infinite set Properties of node and edge property names and Values of property values. We can then define property graphs as follows:

Definition 2.1. A *property graph* is a tuple $G = (V, E, \rho, \lambda, \pi)$ where: (1) $V \subseteq$ Vertices is a finite set of vertex identifiers; (2) $E \subseteq$ Edges is a finite set of edge identifiers; (3) $\rho : E \to (V \times V)$ is a total function mapping edges to ordered pairs of vertices. For convenience, instead of writing $\rho(e) = (v_1, v_2)$, we shall often write $v_1 \xrightarrow{e} v_2$; (4) $\lambda : (V \cup E) \to$ Labels is a total function assigning a label to a vertex or an edge; and (5) $\pi : (V \cup E) \times$Properties $\to$ Values is a partial function mapping an element (edge or vertex) and a property name to a property value.

Example 2.2. Consider the property graph G depicted in Figure 1 modeling a social network, where vertices represent people, and edges represent relationships between them. The labels of vertices is `Person`, while edges are labeled as `follow` or `favorite`. Each person vertex has a `name` property and an `age` property. Each edge has a `since` property indicating the year when the relation started.

Definition 2.3 (Path). A *path* from v to w in a property graph $G = (V, E, \rho, \lambda, \pi)$, where $v, w \in V$, is an alternating sequence $v = v_0 e_1 v_1 e_2, \ldots, e_n v_n = w$ of vertices and edges where $n \geq 0$, such that a path can be a unit vertex v where $n = 0$, or for all $i \in [1, n]$, $e_i \in E$ and $\rho(e_i) = (v_{i-1}, v_i)$ for *forward edge* or $\rho(e_i) = (v_i, v_{i-1})$ for *backward edge*.

Note: a path can be empty, denoted by ϵ.

Regular path queries. Regular path queries (RPQs for short) [2] in a graph database G is an expression of the form $(v, \texttt{regex}, ?x)$, where v is a node in G and `regex` is a regular over the alphabet of edge labels, and $?x$ the output variable. The output of an RPQ over G, denoted $[\![(v, \texttt{regex}, ?x)]\!]_G$, is the set of all nodes v' such that v' can be reached from v by a path P in G and the edge labels along this path form a word accepted by the regular expression `regex`.

The product graph construction. A common way to evaluate RPQs is based on the product graph construction [6, 10]. Given a graph database $G = (V, E, \rho, \lambda, \pi)$ and an RPQ $q = (v, \texttt{regex}, ?\texttt{x})$, the *product graph* is constructed by first converting the regular expression `regex` into an equivalent non-deterministic finite automaton $(Q, \Sigma, \delta, q_0, F)$. Here Q is a finite set of states, Σ a finite alphabet of edge labels, $\delta \subseteq Q \times \Sigma \times Q$ the transition relation, and the initial state is q_0, while F is the set of final states. The product graph $G_\times$ is defined as the graph database $G_\times = (V_\times, E_\times, \rho_\times, \lambda_\times, \pi_\times)$, where (1) $V_\times = V \times Q$; (2) $E_\times = \{(e, (q_1, a, q_2)) \in E \times \delta \mid \lambda(e) = a\}$; (3) $\rho_\times(e, d) = ((x, q_1), (y, q_2))$ if: $d = (q_1, a, q_2)$, $\lambda(e) = a$ and $\rho(e) = (x, y)$; (4) $\lambda_\times((e, d)) = \lambda(e)$; (5) $\pi_\times(v, q) = \pi(v)$; (6) $\pi_\times(e, d) = \pi(e)$. In the final two items we abuse the notation slightly to signal that the set of attributes for nodes or edges is inherited from the original database G. Each node of the form (u, q) in $G_\times$ corresponds to the node u in G and, furthermore, each path P of the form $(v, q_0)(e_1, d_1)(v_1, q_1) \ldots (e_n, d_n)(v_n, q_n)$ in $G_\times$ corresponds to a path $p = ve_1v_1 \ldots e_nv_n$ in G that (a) has the same length as P and (b) brings the automaton from state q_0 to q_n. As such, when $q_n \in F$, then this path in G matches `regex`. In other words, all nodes v' that can be reached from v by a path that matches `regex` can be found by using standard graph search algorithms (e.g., BFS/DFS) on $G_\times$ starting in the node (v, q_0).

3 QUERY LANGUAGE

This section introduces the syntax and semantics of *parametric regular expressions* and *parametric regular path queries*. Parametric regular expressions extend standard regular expressions (used in RPQs and GQL) in two aspects: (1) parametric regular expressions can not only express patterns over edges, but also express patterns over nodes. (2) parametric regular expressions can specify properties both *within* and *beyond* the active domain of a property graph.

In addition to parametric regular expressions, we also introduce *parametric automata*, which serve as an execution model for parametric regular path queries. We also analyze the complexity of evaluating parametric regular path queries over property graphs.

3.1 Parametric Regular Expressions

Syntax. The syntax of parametric regular expressions is based on the syntax of regular expressions in [37], and the formal definition is given below.

Definition 3.1 (Syntax of Parametric Regular Expressions). We assume a set $\mathcal{L}$ of labels, a finite set $\mathcal{P}_n$ of *numerical* properties, a finite set $\mathcal{P}_c$ of *string* properties and a set of *global parameters* Var. The abstract syntax of parametric regular expressions is defined as follows:

$$
\begin{array}{lcll}
E & ::= & (t, \phi) & t \in \mathcal{L} \\
 & \mid & \hat{}E & \textit{inverse} \\
 & \mid & E_1/E_2 & \textit{concatenation of } E_1 \textit{ and } E_2 \\
 & \mid & E_1 \mid E_2 & \textit{alternation of } E_1 \textit{ and } E_2 \\
 & \mid & E^* & \\
 & \mid & E^+ & \\
 & \mid & E^? & \\
\phi & ::= & x = c_{str} & x \in \mathcal{P}_c \textit{ and } c_{str} \textit{ is a string constant} \\
 & \mid & t_{ar} \sim t_{ar} & \sim \in \{>, <, \leq, \geq, \neq, =\} \\
 & \mid & \phi \wedge \phi & \\
t_{ar} & ::= & c & c \in \mathbb{Q} \\
 & \mid & p & \textit{global parameters or properties} \\
 & \mid & c \cdot p & c \in \mathbb{Q} \\
 & \mid & t_{ar} + t_{ar} & \\
p & ::= & attr & attr \in \mathcal{P}_n \\
 & \mid & ?x & ?x \in Var
\end{array}
$$

For the convenience of presentation, we call $attr \in \mathcal{P}_n$ as *numerical attributes*, and $attr \in \mathcal{P}_c$ as *string attributes*.

Example 3.2. Let us review the example from introduction: consider the property graph in example 2.2, and suppose now that we wish to find paths labelled by follows such that the link is rather new (e.g. established after 2021), but also that the maximum difference between the age of two people along this paths is no more than s even years. The above property can be formalize into the following parametric regular expression r_f:

$$
\begin{aligned}
&((\texttt{Person}, ?p \leq age \wedge ?q \geq age \wedge ?q - ?p \leq 7)/ \\
&(\texttt{follow}, since > 2021))^+/ \\
&(\texttt{Person}, ?p \leq age \wedge ?q \geq age \wedge ?q - ?p \leq 7)
\end{aligned}
$$

Semantics. We start with the *sequence of elements*, which is what can be captured by a parametric regular expression. Intuitively, each element contains a label and assignments to some attributes, such as $(\texttt{Person}, [name \mapsto \text{'Alice'}, age \mapsto 25])$. Formally, we assume a countably infinite set Labels of *forward* labels, Properties of properties names and Values of properties values. For each forward label $l \in$ Labels, we have a related *inverse label* $\hat{}l$, and the set of inverse labels is denoted as $\hat{}$Labels An element is a pair (l, f), such that $l \in$ Labels$\cup\hat{}$Labels and f : Properties $\mapsto$ Values. We denote a set of elements as Σ. A *sequence of elements* is a finite list p such that $p \in \Sigma^*$. Given two sequences $p_1, p_2 \in \Sigma^*$, where $p_1 = (l_1, f_1), \ldots, (l_n, f_n)$ and $p_2 = (l'_1, f'_1), \ldots, (l'_n, f'_n)$, the *concatenation* of p_1 and p_2 is a new sequence $p_1 \cdot p_2 = (l_1, f_1), \ldots, (l_n, f_n), (l'_1, f'_1), \ldots, (l'_n, f'_n)$.

Interpretation of Data Constraints. The data constraints in parametric regular expressions are interpreted in terms of *assignments* to global parameters and the theory of Quantifier-Free Linear Real Arithmetic and the theory of equality (with string constants) [4] which are supported by major SMT solvers like Z3 [7]. In the following, we denote the theory of Quantifier-Free Linear Real Arithmetic and equality as $\mathfrak{T}$.

A global parameters assignment is a total function that assigns each global parameter to a rational value, and we write $\emptyset$ for empty assignments. Given two assignments μ and μ', we say that μ and μ' are *unifiable* if $\mu(x) = \mu'(x)$ for all $x \in dom(\mu) \cap dom(\mu')$, and we define the *unification* $(\mu \cup \mu')$ of two unifiable assignment μ, μ' $(\mu \cup \mu')(x) = \mu(x)$ if $x \in dom(\mu)$, and otherwise $(\mu \cup \mu')(x) = \mu'(x)$.

Definition 3.3 (Semantics of Data Constraint). Given an element (l, f), an assignment to global parameters μ and a data constraints $\phi(\overline{x}, \overline{p})$ where $\overline{x}$ are global parameters and $\overline{p}$ are properties variables. $\mathfrak{T} \models \phi(\mu(\overline{x}), f(\overline{p}))$ is defined inductively over ϕ as follows:

- $\mathfrak{T} \models \theta_1 \wedge \theta_2$ iff $\mathfrak{T} \models \theta_1$ and $\mathfrak{T} \models \theta_2$.
- $\mathfrak{T} \models f(p) = c_{str}$ iff p is a string property variable and $f(p) = c_{str}$.
- $\mathfrak{T} \models \theta(\mu(\overline{x}), f(\overline{p}))$ iff $\overline{p}$ are numeric properties variables and $\theta(\overline{x}, \overline{p})$ is an (in)equality over linear terms over rational variables.

Definition 3.4 (Language of Parametric Regular Expression). The *language* of a parametric regular expression r with an assignment μ is a set of sequences over Σ, , denoted as $[[r]]^{\mu} \subseteq \Sigma^*$. $[[r]]^{\mu}$ is defined inductively as follows:

$$
\begin{aligned}
[[\epsilon]]^{\emptyset} &= \{\epsilon\} \\
[[(l, \phi(\overline{x}, \overline{p}))]]^{\mu} &= \left\{(l, f) \,\middle|\, \mathfrak{T} \models \phi(\mu(\overline{x}), f(\overline{p}))\right\} \\
[[\hat{}r]]^{\mu} &= \left\{(\hat{}l, f) \,\middle|\, (l, f) \in [[r]]^{\mu}\right\} \\
[[r_1/r_2]]^{\mu_1 \cup \mu_2} &= \left\{(a_1 \cdot a_2) \,\middle|\, a_1 \in [[r_1]]^{\mu_1}, a_2 \in [[r_2]]^{\mu_2}\right\} \\
[[r_1 \mid r_2]]^{\mu_1 \cup \mu_2} &= \left\{a \,\middle|\, a \in [[r_1]]^{\mu_1} \cup [[r_2]]^{\mu_2}\right\}
\end{aligned}
$$

Moreover, assuming that $r^1 = r$ and $r^{n+1} = r^n/r$ for every $n \geq 1$, we have:

$$
[[r^*]]^{\emptyset \cup \mu_1 \cup \cdots \mu_i} = [[\epsilon]]^{\emptyset} \cup \bigcup_{n \geq 1} [[r^n]]^{\mu_i}
$$

where each $\mu_i, \mu_j, 0 \leq i, j \leq n$ are pairwise unifiable.

The definition of r^+ can be derived from r/r^*, and $r^?$ can be derived from $r \mid \epsilon$.

Definition 3.5 (Membership of Parametric Regular Expressions). We say that r accepts an element sequence e if there exists an assignment μ, such that $e \in [[r]]^{\mu}$. The acceptance relation is denoted by $p \in \mathcal{L}(r)$

Example 3.6. We continue consider the parametric regular expression at Example 3.2. Consider the following sequence ele

$$
\underbrace{(\texttt{Person}, \overbrace{[age \mapsto 25]}^{fe_1})}_{e_0}, \underbrace{(\texttt{follow}, \overbrace{[since \mapsto 2022]}^{fe_2})}_{e_1}, \underbrace{(\texttt{Person}, \overbrace{[age \mapsto 28]}^{fe_3})}_{e_2}
$$

We say $ele \in [[r_f]]^{\mu}$ with μ :$?p \mapsto 25, ?q \mapsto 28$

Semantics over Property Graphs. The semantics of parametric regular expressions is coherent with the semantics of path patterns over property graphs [37]. We take each path p over a property graph $G = (V, E, \rho, \lambda, \pi)$ as a sequence of elements $(l_1, \pi'_1), \ldots, (l_n, \pi'_n)$, where for each $p_i \in E \cup V$, $l_i = \lambda(p_i)$, and π' : Properties $\mapsto$ Values is a partial function such that $\pi'(a) = \pi(p_i, a)$. Note the inverse-labeled element in a path p are interpreted as nodes or inverse edges over G, i.e. if $p_i \in p$ such that $p_i = (l_i, \pi'_i)$ and $l_i = \hat{}l$, then either $p_i \in V$ and $l = \lambda(p_i)$, or $p_i = \rho(p_{i+1}, p_{i-1})$.

The *set of answers* of a parametric regular expression r over a property graph $G = (V, E, \rho, \lambda, \pi)$, denoted as $[[r]]_G$ is a set of paths, such that each path $p \in \mathcal{L}(r)$.

3.2 Parametric Regular Path Queries

Definition 3.7 (Parametric Path Regular Queries). A parametric regular path query (PRPQ for short) over a property graph $G = (V, E, \rho, \lambda, \pi)$ is an expression of the form $q = (v, pregex)$, with $v \in V$, and $pregex$ a *parametric regular expression*. The query q over a property graph $G = (V, E, \rho, \lambda, \pi)$ returns `true` if there exists a node v', a path p from v to v', such that $p \in [[pregex]]_G$. Otherwise the query returns `false`.

Example 3.8. Let us reconsider Example 2.2. If we start the query from 'Alice' and query with the parametric regular expression r_f Example 3.2, we can formalize the query as PRPQ(Alice, r_f). The query result is `true` with the path and Alice $\xrightarrow{\texttt{follow}}$ Charlie and assignments $\{?p \mapsto 25, ?q \mapsto 28\}$. However, if the query starts from the node 'Bob' i.e. PRPQ(Bob, r_f), the query result is `false`.

3.3 Parametric Automaton

We propose *parametric automaton* as our execution model inspired by sequence theory [19] which decide the constraints over a sequence of objects with data domains. Parametric automata take an input path as a sequence of objects with attributes, capture object patterns and evaluate data conditions and find a model for global parameters. We will show that parametric automata have the same computation power as parametric regular expressions.

Definition 3.9 (Parametric Automaton). A parametric automaton is a tuple $(\mathsf{L}, \chi, Attr, Q, Q_0, F, \Delta)$, where: (1) $\mathsf{L} \subseteq$ Labels is a set of symbols (2) χ is a set of global variables. (3) $Attr \subseteq$ Properties is a set of properties. (4) Q is a finite set of states (5) $Q_0 \subseteq Q$ is a set of start states. (6) $F \subseteq Q$ is a set of final states. (7) We define the transitions Δ as follows: $\Delta \subseteq Q \times (\Sigma \times T(\chi, Attr) \times \mathbb{B}) \times Q$ where $T(\chi, Attr)$ is the set of string and linear arithmetic formulas over $\chi \cup Attr$ following the format of Definition 3.1, and $\mathbb{B}$ is a boolean value that indicates the transition is inverse if it is true and vice versa. If $(q, (\sigma, \phi, inv), q') \in \Delta$, where $\sigma \in \mathsf{L}, \phi \in T(\chi, Attr)$, then we write $q \xrightarrow{(\sigma, \phi, inv)} q'$.

Definition 3.10 (Acceptance Conditions of Parametric Automaton). A parametric automaton $Aut = (\mathsf{L}, \chi, Attr, Q, Q_0, F, \Delta)$. *Aut accepts* an element sequence $(l_1, f_1), \ldots, (l_n, f_n) \in \Sigma^*$ with respect to an assignment μ, if there is a sequence of transitions

$$
q_0 \xrightarrow{(\sigma_1, \phi_1, inv_1)} q_1 \xrightarrow{(\sigma_2, \phi_2, inv_2)} q_2 \ldots \xrightarrow{(\sigma_n, \phi_n, inv_n)} q_n
$$

such that: (1) $q_0 \in Q_0, q_n \in F$ (2) $\forall i \in \{1, \ldots, n\}, \sigma_i = l_i$; (3) $\forall i \in \{1, \ldots, n\}, \mathfrak{T} \models \phi_i(\mu(\overline{x}), f_i(\overline{p}))$. (4) if inv_i is true, l_i should be an *inverse* label, and otherwise l_i should be a *forward* label.

Example 3.11. The parametric regular expression in Example 3.2 can be formalized as the following parametric automaton *Aut* depicted in Figure 2, where $\overline{x} = \{?p, ?q\}$, $\overline{p} = \{age, since\}$ and $\varphi(\overline{x}, \overline{p})$ =$?p \leq age \wedge ?q \geq age \wedge ?q - ?p \leq 7$, and the inverse labels are all `true` and omitted for simplicity.

In Example 3.6, the sequence $ele \in [[r_f]]^{\mu}$ where μ :$?p \mapsto 25, ?q \mapsto 28$, and it is straightforward to verify that the parametric automaton *Aut* also accepts *ele* with the same assignment μ by the

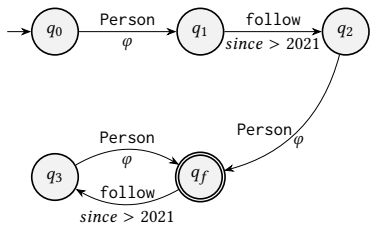


Figure 2: Parametric automaton example.

following run:

$$q_0 \underbrace{\xrightarrow[\texttt{Person}]{\mathfrak{T}\models\varphi(\mu(\overline{x}),f_{e1}(\overline{p}))}}_{\text{visit } e_1} q_1 \underbrace{\xrightarrow[\texttt{follow}]{\mathfrak{T}\models f_{e2}(since)>2021}}_{\text{visit } e_2} q_2 \underbrace{\xrightarrow[\texttt{Person}]{\mathfrak{T}\models\varphi(\mu(\overline{x}),f_{e3}(\overline{p}))}}_{\text{visit } e_3} q_f$$

Each parametric regular expression can be converted to a parametric automata. The following conversion is a variant of a standard regular expression to NFA conversion. Formally, we have the following theorem:

Theorem 3.12. *For each parametric regular expressions r, there is a parametric automaton $\mathcal{A} = (\Sigma, \chi, Attr, Q, Q_0, F, \Delta)$ with a single initial state such that for each $p \in [[r]]^{\mu}$, r is accepted by $\mathcal{A}$ with assignment μ.*

The complete proof is leaved in appendix.

3.4 The Hardness of Parametric Regular Path Queries

Although the data complexity (i.e. the complexity of evaluating a fixed PRPQ$(v, pregex)$ query over a property graph G) is NL-complete [12, 31], the combined complexity (where both the query and G are part of the input) is NP-hard *in general*. This can be shown by a reduction from the Boolean satisfiability problem (3-SAT).

Theorem 3.13. *The combined complexity of parametric regular path queries over a property graph is* NP*-hard.*

Proof. We reduce from the Boolean satisfiability problem (3-SAT). Given a Boolean formula ϕ in CNF with variables $x_1, x_2, \ldots, x_n$ and clauses $C_1, C_2, \ldots, C_m$, we construct a property graph G and a PRPQ$(v, pregex)$ query such that ϕ is satisfiable if and only if there exists a path in G.

We construct G with a single vertex v labeled by v with a single property a such that $\pi(v, a) = 1$, and a single self loop edge e labeled by e with no properties and $\rho(e) = (v, v)$.

We construct a parametric regular expression based on the formula ϕ *inductively*, with a set of rational global parameters $\{?x_1, \ldots, ?x_n\}$. The base case starts from the first clause C_1, where we construct a parametric regular expression r_1 as follows:

$$r_1 = (v, ?x_i \sim_i 0) \mid (v, ?x_j \sim_j 0) \mid (v, ?x_k \sim_k 0)$$

if $?x_i$ is positive in C_1, then $\sim_i$ is $\neq$ otherwise $\sim_i$ is $=$. The cases for $?x_j, ?x_k$ are similar.

If we encode $C_1, \ldots C_i$ by a parametric regular expression r_i, we construct r_{i+1} for clause C_{i+1} as follows:

$$\begin{aligned} r_{i+1} = r_i/(((e, true)/(v, ?x_i \sim_i 0) \\ \mid (e, true)/(v, ?x_j \sim_j 0) \\ \mid (e, true)/(v, ?x_k \sim_k 0))) \end{aligned}$$

where $\sim_i, \sim_j, \sim_k$ are defined as in the base case.

According to the Definition 3.7, we construct the parametric regular path query as PRPQ(v, r_m), and if there exists a path in G that matches r_m, then the corresponding assignment of global parameters $\{?x_1, \ldots, ?x_n\}$ in r_m also satisfies all clauses in ϕ. Conversely, if ϕ is satisfiable, the model of all clauses provides an assignment to the global parameters $\{?x_1, \ldots, ?x_n\}$ in r_m, and then there exists a path in G that matches r_m.

Therefore, we have reduced 3-SAT to the problem of evaluating a PRPQ$(v, pregex)$ query over a property graph, proving that the combined complexity is NP-hard. □

4 NAIVE EVALUATION ALGORITHM

According to our execution model and the product graph construction, there is a straight-forward naive algorithm to evaluate parametric regular path queries over property graphs. We can perform breadth-first search (BFS) on the product of the parametric automaton and the property graph and accumulates all instantiated data constraints along the paths. Whenever a final state in the automaton is reached, we check the satisfiability of the accumulated constraints by an SMT solver. The intuitive idea can be illustrated by the following example.

Example 4.1 (Naive Evaluation of a Parametric Regular Path Query). Let's consider the query PRPQ(Alice, r_f) in Example 3.8 over the social network in Figure 2.2, where r_f has been translated into the parametric automaton in Figure 2.

The evaluation starts from node 'Alice' and the initial state q_0 in the automaton. We visit the node 'Alice' and make a transition on the automaton with the label '`Person`' and advances to state q_1. Now we obtain an instantiated data constaint from the transitio $q_0 \xrightarrow[\text{'Person'}]{\phi} q_1$ with the property values $age = 25$ of node 'Alice', such that $?p \leq 25 \wedge ?q \geq 25 \wedge ?q - ?p \leq 7$.

Then the algorithm expands the neighbor edges from 'Alice' to 'Charlie' via edge 'follow'. We makes an transition on the automaton with the label '`follow`', and advances to state q_2, and obtain an instantiated data constraint $2022 > 2021$.

After that, we visit the node 'Charlie' and make a transition on the automaton with the label '`Person`', and advances to state q_f. We obtain another instantiated data constraint $?p \leq 28 \wedge ?q \geq 28 \wedge ?q - ?p \leq 7$ from the transition $q_2 \xrightarrow[\text{'Person'}]{\phi} q_f$ with the property values of node 'Charlie'.

Finally, we check the satisfiability of the following conjunction of

$$\underbrace{?p \leq 25 \wedge ?q \geq 25 \wedge ?q - ?p \leq 7}_{\text{data constraints from 'Alice'}} \wedge \quad \underbrace{2022 > 2021}_{\text{data constraints from ''follow'}} \wedge \underbrace{?p \leq 28 \wedge ?q \geq 28 \wedge ?q - ?p \leq 7}_{\text{data constraints from 'Charlie'}} \tag{1}$$

which is satisfiable with the assignments $\{?p \mapsto 25, ?q \mapsto 28\}$, and thus we detect a valid path Alice $\xrightarrow{\texttt{follow}}$ Charlie as an answer to the query.

4.1 Naive Algorithm

Although the intuition is straightforward, there is one challenge to formalize the naive algorithm, which is that parametric regular expressions specify patterns on both nodes and edges. The formalization of the product graph construction can not be directly applied to our setting, because we need to formalize the behaviour of the algorithm when it encounters a new node or edge in the property graph, and how the algorithm advances the automaton state and accumulates data constraints accordingly. We call such behaviour as *synchronized transitions*.

We provide two helper functions, TransNode and TransEdge, to formalize the transitions on nodes and edges. The TransNode function formalizes making transitions on a node, which scan all transitions originating from the current automaton state q in $\mathcal{A}$, matches with the given node v or edge e in property graph G, and returns all satisfied successor states q' (or pairs of successor states and nodes). The TransEdge function is similar to the pattern matching of TransNode, the main difference is the matching of edges considers the directionality of edges in G by analyzing the *inv* flag in transitions. If the *inv* flag is $\top$, the function collects all edges e' *originate from* v with $\lambda(e) = l$, and returns with successor states and the destination node v' pairwise. If the *inv* flag is $\bot$, the function collect all edges e' *enter into* v with $\lambda(e) = l$, and returns with successor states and the source node v pairwise.

$$\textsc{TransNode}(v, q, G, \mathcal{A}) = \{q' \mid q \xrightarrow{(l,\varphi,inv)} q', l = \lambda(v)\}$$

$$\textsc{TransEdge}(e, q, G, \mathcal{A}) = \begin{cases} \{(v', e', q') \mid q \xrightarrow{(l,\varphi,\top)} q', \hat{}l = \hat{}(\lambda(e')), v \xrightarrow{e'} v'\} \\ \{(v', e', q') \mid q \xrightarrow{(l,\varphi,\bot)} q', l = \lambda(e'), v' \xrightarrow{e'} v\} \end{cases}$$

The framework of synchronized transitions can be summerized as following: (1) At the beginning of the exploration, the algorithm advances with the pattern and properties value of start node on the automaton, but not step into the neighbor edges. (2) During the exploration, when the algorithm *expands* neighbors and visits a new edge e, the algorithm advances according to v's the pattern and the properties values, and then immediately follows a transition according to the patterns and properties values of the v's destination node..

We propose the naive evaluation algorithm in the NaiveQuery procedure of Algorithm 1 following the formalization of PathFinder [11]. Algorithm 1 record the search information in *search states*.

Definition 4.2. A search state is a tuple $(v, q, e, prev, \mathcal{F})$ where: (1) $v \in V$ is the current graph node (2) $q \in Q$ is the current automaton state (3) $e \in E$ is the current graph edge. (4) *prev* is a pointer to the previous search state (5) $\mathcal{F}$ contains formulas with attribute variables instantiated using actual property values from visited nodes/edges

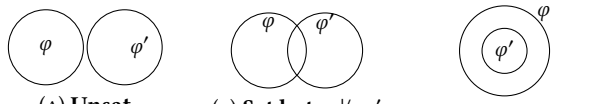
The algorithm also maintains two data structures: (1) Open, which is a queue of search states, with usual queue operations (enqueue, dequeue). (2) Visited, which is a dictionary of search states using tuples (v, e, q) as keys, that have already been explored, maintained to avoid infinite loops,

The naive query algorithm follows 'SIMPLE' mode path semantics, i.e. the algorithm explores paths such that nodes are not repeated except the start point. This condition is added because it is not make sense to visit a node more than twice in real world query, and is decided by the IsValid function [11].

Algorithm 1 begins exploration from the initial state (v, q_0). It checks whether a trivial path only containing $v \in V$ constitutes a valid answer (Lines 5-7), and such case does not require a model of global parameters. The algorithm then performs BFS over the property graph with the synchronization transitions framework with the start node as well as in the main loop of BFS. If the automaton state q' is a final state, the algorithm checks whether the accumulated formulas $\mathcal{F}$ are satisfiable. If these conditions are satisfied, the algorithm constructs the path by the GetPath procedure via backtracking on the *prev* domain of search states, which is a variant of GetPath function in Algorithm 1 of [11], and constructs a model of the global parameters by querying an SMT solver, and then the algorithm returns `true` with the path and the model. When the 'Open' queue is exhausted, and no answer is detected, then the algorithm returns (`false`, $\emptyset$).

Algorithm 1 Query evaluation of PRPQ$(v, pregex)$ over a property graph $G = (V, E, \rho, \lambda, \pi)$.

1: **procedure** NaiveQuery($regex, G, v$)
2: $\mathcal{A} \leftarrow Automaton(regex)$ ▷ q_0 initial state, F final states
3: Open.init()
4: Visited.init()
5: **if** $v \in V$ **and** $q_0 \in F$ **then**
6: **return**(`true`, $(v, \emptyset)$) ▷ No need for a model
7: **for all** $q' \in$ TransNode$(v, q_0, G, \mathcal{A})$ **do**
8: $\mathcal{F} \leftarrow \emptyset \cup \{\varphi[attr/\pi(v, attr)] \mid q_0 \xrightarrow{(\lambda(v),\varphi,inv)} q'\}$
9: start_state $\leftarrow (v, q', \bot, \bot, \mathcal{F})$
10: Open.enqueue(start_state)
11: **while** Open $\neq \emptyset$ **do**
12: state $= (v, q, e, prev, \mathcal{F}) \leftarrow$ Open.dequeue()
13: **for all** $next = (v', e', q') \in$ TransEdge$(e, q, G, \mathcal{A})$ **do**
14: **if** not isValid(state, next) **then**
15: continue ▷ Skip not simple path
16: $\mathcal{F}' \leftarrow F \cup \{\varphi[attr/\pi(e', attr)] \mid q \xrightarrow{(\lambda(e'),\varphi,inv)} q'\}$
17: **for all** $q'' \in$ TransNode$(v', q', G, \mathcal{A})$ **do**
18: $\mathcal{F}'' \leftarrow F' \cup \{\varphi[attr/\pi(v', attr)] \mid q' \xrightarrow{(\lambda(v'),\varphi,inv)} q''\}$
19: nextState $\leftarrow (v', q'', e', curr, \mathcal{F}'')$
20: **if** $(v', q'', e', *, *) \notin$ Visited **then**
21: Visited.add(nextState)
22: Open.enqueue(nextState)
23: **if** $q'' \in F$ **and** CheckSat$(\mathcal{F}'')$ **then**
24: path $\leftarrow$ GetPath(nextState)
25: $\mu \leftarrow$ GetModel$(\mathcal{F}'')$
26: **return** (`true`, (path, μ))
27: **return** (`false`, $\emptyset$) ▷ No path detected



(A) **Unsat** (B) **Sat but $\varphi \not\models \varphi'$** (C) **Sat with $\varphi \models \varphi'$**

Figure 3: The assignments of φ and φ'.

4.2 Complexity Analysis

Although the naive algorithm can evaluate parametric regular path queries correctly, it has an exponential data complexity $O(2^{|E|+|V|})$ with a property graph $G = (V, E, \rho, \lambda, \pi)$. The algorithm accumulates $O(|E| + |V|)$ instantiated formulas if we fix the size of the query and solving such a formula is exponential in the worst case.

5 OPTIMIZED QUERY EVALUATION

Although Theorem 3.13 states that the combined complexity of parametric regular path queries is NP-hard, it is still possible to achieve *exponential acceleration* in terms of data complexity. We propose a new data structure called *macro state* to prevent duplicated data constraints and also propose a macro-state based query evaluation algorithm to achieve exponential acceleration in terms of data complexity.

5.1 Macro States

The search state in Definition 4.2 contains a conjunction of instantiated data constraint. According to the syntax of parametric regular expression in Definition 3.4, these data constraints contain: (1) ground formulas, which are from data constraints *only* over properties and can be directly evaluated. (2) linear (in)equalities over global parameters.

Regarding the (in)equalities over global parameters, we focus on the *assignments* of the global parameters. Assume we have accumulated a set of arithmetics constrants φ, and here comes a new set of (in)equalities φ'. The relationship between the assignments of φ and φ' can be categorized into three cases, as shown in Figure 3: (1) The assignments of φ and φ' are disjoint as Figure 3a which means *unsatisfiable*. (2) The assignments of φ and φ' are intersecting as Figure 3b, which means $\varphi \wedge \varphi'$ is satisfiable, but $\varphi \not\models \varphi'$. In this case, we need to record φ' to obtain new assignments $\mathfrak{A}$ such that each $\mu \in \mathfrak{A}, \mu \models \varphi$ and $\mu \models \varphi'$. (3) The assignments of φ' is included in the assignments of φ as Figure 3c, which means $\varphi \models \varphi'$. In this case, we do not need to record φ'.

However, in practice, it is not efficient to query the oracle to decide if a new encountered (in)equality is entailed by the existing constraints, because the linear programming solver has a quadratic time complexity in the number of constraints. We can optimize the process by *compare the bounds of terms over global parameters.* These linear (in)equalities over global parameters can be rewrite into *affine forms* over global parameters, i.e. $\sum a_i p_i \sim c, a_i, c \in \mathfrak{R}, p_i \in Var$, which are also b ounds of terms over global parameters. Note, for a given parametric automaton, the numbers of thse terms are fixed, and are smaller or equal than the number of atomic data constraints of the parametric automaton.

This can be derived from the linearity of arithmetic constraints. For each atomic (in)equality in a parametric automaton, we can rewrite the (in)equality into the following form $\sum a_i x_i \sim b_i p_i + c$, such that $x_i \in Var$, $p_i \in$ Properties, and $a_i, b_i, c \in \mathbb{Q}$, which can be an affine form after instantiating property values. The left hand side of the affine (in)equality is a linear term over global parameters, and the number of such linear terms is fixed and smaller or equal than the number of atomic data constraints in the parametric automaton.

As a result, when we encounter a new (in)equality in affine form, we can compare the bounds of the corresponding term with the existing bounds. If the new (in)equality is not entailed by the existing bounds, we update the bounds. Otherswise, we can skip the new (in)equality because it is entailed. If the corresponding term has not appeared, we add it to the set of bounds directly. We only check the consistency of bounds when the bounds are updated, so that we avoid query the oracle by *comparision in* $O(1)$ and can only query the oracle *once* after visiting a node or an edge in the property graph, which significantly reduces the number of queries to the oracle.

Let's reconsider the exploration of query PRPQ(Alice, r_f) from Example 3.8, which is shown in Figure 4.

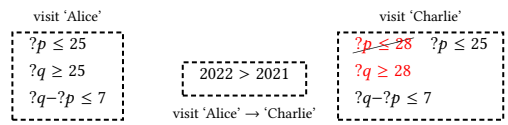


Figure 4: The exploration of query PRPQ(Alice, r_f) with macro states.

After visiting 'Alice', we record three bounds $?p \leq 25$, $?q \geq 25$ and $?q{-}?p \leq 7$, and these bounds are consistent with an assignment μ_1. After visiting the edge 'Alice' → 'Charlie', the groud formula $2022 > 2021$ is evaluated directly. After visiting 'Charlie', there are three new instantiated bounds: $?p \leq 28$, $?q \geq 28$ and $?q{-}?p \leq 7$. $?q{-}?p \leq 7$ is unchanged. while we do not have to update $?p \leq 28$, because $\mu_1 \models ?p \leq 28$. However, we do have to update $?q \geq 28$ because $\mu_1 \not\models ?q \geq 28$. After the update, we can check the consistency of bounds by querying the oracle, which is still consistent with a new assignment μ_2. The consistency of bounds indicates that there is no inconsistency in the accumulated formula, and we finish the exploration successfully.

We formalize the above intuition by defining a new data structure called *macro state* as follows.

Definition 5.1 (Macro State). A macro state is a tuple $(q, e, v, prev, up, low, neq)$ where: (1) q , e , v , $prev$ are the same as in a search state according to Definition 4.2. (2) up stores the upper bounds of terms. Formally, for $\sum a_i p_i \leq c$, where $a_i, c \in \mathbb{Q}$ and $p_i \in \chi$, if and only if $up\,[\sum a_i p_i] = c$ (3) low stores the lower bounds of terms. Formally, for $\sum a_i p_i \geq c$, where $a_i, c \in \mathbb{Q}$ and $p_i \in \chi$, if and only if $low\,[\sum a_i p_i] = c$ (4) neq handles the $\neq$ operand. Formally, for $\sum a_i p_i \neq c$, where $a_i, c \in \mathbb{Q}$ and $p_i \in \chi$, if and only if $neq\,[\sum a_i p_i] = c$

However, there is still one challenge to address:it is only feasible to maintain upper and lower bounds for (in)equalities with $\leq$ and $\geq$ operators, but not for $<$, $>$ and $=$ operators. For this challenge, we introduces a helper constant c_h such that $c_h > 0$, and rewrite the (in)equalities of each data constraint into *equal-satisfied*

(in)equalities by applying the following rules:

$$\begin{aligned} t_l < t_r &\mapsto t_l + c_h \leq t_r \\ t_l > t_r &\mapsto t_l - c_h \geq t_r \\ t_l = t_r &\mapsto t_l \leq t_r \wedge t_l \geq t_r \end{aligned} \tag{2}$$

Note: we add $c_h > 0$ to the oracle as a *precondition* when the oracle checks the consistency of bounds. The rewriting introduces $O(n)$ new (in)equalities over linear terms over rational variables, where n is the number of (in)equalities in original formula.

Algorithm 2 updates a macro state and returns $\top$ as long as the updated bounds are consistent. It check the consistency of bounds by querying an *oracle* at the CheckBounds function.

Note that in Algorithm 2 queries the oracle by the following steps: it adds the predcondition $c_h > 0$ to an oracle, and translates the bounds (up, low, neq) into inequalities according to Definition 5.1 and adds these (in)eqaulities to the oracle, and then check the satisfiability of the combined constraints. The satisfiable results mean the updated bounds are consistent, and vice versa.

Algorithm 2 Update the bounds of terms (up, low, neq) with an edge or a node o from a property graph and a conjunction of data constraint φ.

```
procedure Update(φ, o, (up, low, neq))
    φ' ← φ[attr/π(o, attr)]
    for all atom ∈ φ' do                    ▷ φ' is a conjunction of atoms
        if atom is a ground constraint and not satisfiable then
            return ⊥, ∅                     ▷ Inconsistency detected
        Normalize atom into affine form, i.e. ∑ a_i p_i ~ c where ~∈ {≤, ≥, ≠}
        if ~ is ≤ then
            if ∑ a_i p_i ∉ up or c ≤ up[∑ a_i p_i] then
                up[∑ a_i p_i] ← c
        else if ~ is ≥ then
            if ∑ a_i p_i ∉ low or c ≥ low[∑ a_i p_i] then
                low[∑ a_i p_i] ← c
        else if ~ is ≠ then
            neq[∑ a_i p_i] ← c
    if CheckBounds(up, low, neq) then
        return ⊤, (up, low, neq)            ▷ Update successful
    else
        return ⊥, ∅                         ▷ Inconsistency detected
```

5.2 Optimized Query Algorithm

Algorithm 3 shows the optimized query evaluation algorithm. It leverage the synchronized transitions framework as the naive algorithm, where the TransNode and TransEdge functions are the same as the naive algorithm. The algorithm can also construct a path and an assignment of global parameters when a final state is reached.

Different from the naive algorithm, Algorithm 3 preprocess data constraints by applying the rewriting rules in (2) before the beginning of exploration, and an exploration is only continued if the update of macro state is successful.

Algorithm 3 Optimized Query Evaluation of PRPQ$(v, pregex)$ over a property graph $G = (V, E, \rho, \lambda, \pi)$

```
procedure OptimizedQuery(regex, G, v)
    𝒜 ← Automaton(regex)                    ▷ q_0 initial state, F final states
    Rewrite all data constraints in 𝒜 according to (2)
    Open.init()
    Visited.init()
    if v ∈ V and q_0 ∈ F then
        return(true, (v, ∅))
    for all q' ∈ TransNode(v, q_0, G, 𝒜) do
        init, (up_0, low_0, neq_0) ← Update(φ, v, (∅, ∅, ∅))   ▷ where q_0 --λ(v),φ--> q
        if init then
            start_state ← (v, q', ⊥, ⊥, up_0, low_0, neq_0)
            Open.enqueue(start_state)
    while Open ≠ ∅ do
        state = (v, q, e, prev, up, low, neq) ← Open.dequeue()
        for all next = (v', e', q') ∈ TransEdge(e, q, G, 𝒜) do
            if not IsValid(state, next) then
                continue                    ▷ skip no simple path
            flag, (up_1, low_1, neq_1) ← Update(φ_1, e', (up, low, neq))   ▷ where q --λ(e'),φ_1--> q'
            if not flag then
                continue                    ▷ inconsistency detected
            for all q'' ∈ TransNode(v', q', G, 𝒜) do
                f', (up_2, low_2, neq_2) ← Update(φ_2, v', (up_1, low_1, neq_1))   ▷ where q --λ(v'),φ_2--> q'
                if not f' then
                    continue                ▷ inconsistency detected
                nextState ← (v', q'', e', curr, up_2, low_2, neq_2)
                if (v', q'', e', *, *, *, *) ∉ Visited then
                    Visited.add(nextState)
                    Open.enqueue(nextState)
                    if q'' ∈ F then
                        path ← GetPath(nextState)
                        μ ← GetModel(up_2, low_2, neq_2)
                        return(true, (path, μ))
    return (false, ∅)                       ▷ No path detected
```

5.3 Complexity Analysis

Algorithm 3 queries the linear programming solver $O(2^c \cdot |A| \cdot (|V| + |E|))$ times, where $\mathcal{A}$ is a parametric automaton with c (in)equality constraints. The 2^c factor arises because each of the c constraints can be independently either present or absent in a macro state, and the algorithm must explore all such combinations in the worst case.

However, as the size of query is typically much smaller than the size of the property graph in practical scenarios, we can analyze in terms of *data complexity*. The optimized algorithm has a linear time complexity in the size of the property graph, i.e., $O(|E| + |V|)$, while the naive algorithm has exponential time complexity in the size of the property graph. The optimized algorithm significantly reduces the complexity compared to the naive approach.

6 EXPERIMENTAL EVALUATION

In this section, we present an experimental evaluation of parametric regular path queries using the algorithms presented in Section 4 and

Section 5. Given that, to the best of our knowledge, no other system supports parametric regular path queries, we focus on showing the efficiency of our algorithms over real world graphs. In particular, we focus on showing that the optimized version of our approach, presented in Algorithm 3 is a *feasible* solution over large graphs, and that its macro-state based optimization achieves significant improvement of the baseline solution given in Algorithm 1. For this, we focus on the following research questions:

RQ1 How does Algorithm 3 scale with graph size and density?
RQ2 How does the performance vary with different queries depending both on regular expressions and data constraints?
RQ3 How does the oracle query affect the overall performance?
RQ4 What is the performance improvement of the optimized algorithm (Algorithm 3) over the naive algorithm (Algorithm 1)?

6.1 Experiment Setup

Implementation. We implement parametric regular path queries and their evaluation algorithms in MillenniumDB [37], an open-source persistent graph database system written using the `C++` programming language. We use the Z3 SMT solver [7] as oracle for queries. The syntax of parametric regular expression is embedded into the MQL query syntax used in MillenniumDB, which resembles Cypher and GQL. Both the naive query algorithm and the optimized query algorithm are incorporated into the standard execution pipeline of MillenniumDB.

Datasets. Table 1 lists the characteristics of the labeled-graph dataset used in the experiments. The ICIJ-Leaks and ICIJ-Paradises datasets [27, 28] contain information about offshore entities and their relationships, which have natural attributes and labels on both nodes and edges. These datasets are relatively sparse. The LDBC01 and LDBC10 datasets are originally used for graph database benchmarking [25], which are synthetic datasets and simulate a social network with various types of nodes and edges. These datasets are relatively densely connected, and the LDBC10 dataset is significantly larger than the other datasets used in the experiments. The Pokec dataset [32] is a social network. Nodes represent users and contain many attributes including age, gender, and location, while edges represent relations between users. Labels on nodes and edges are generated synthetically. Pokec dataset has a large size and higher density. The Telecom dataset [22] contains the relationships between users and behaviors in a telecom network, which has natural attributes and labels on both nodes and edges. This dataset is also the most dense graph among the datasets we used.

Table 1: Dataset Characteristics.($1\,\mathrm{M} = 10^6, 1\,\mathrm{K} = 10^3$)

Name	Dataset	$\lvert V\rvert$	$\lvert E\rvert$	$\lvert Le\rvert$	$\lvert Lv\rvert$	$\frac{\lvert E\rvert}{\lvert V\rvert}$
IL	ICIJ-Leaks	1.9M	3.2M	14	5	1.68
IP	ICIJ-Paradises	163K	364K	6	5	2.23
L0	LDBC01	180K	768K	8	15	4.17
L1	LDBC10	30M	178M	9	15	5.93
PO	Pokec	1.6M	30.6M	3	1	18.7
TE	Telecom	170K	50M	3	4	294

Query Generation. Query templates in our experiments consist of two components: *regular path templates* which are regular expressions that represent patterns of paths, and *data constraint templates* that describe relations between objects along a path. A *query template* is instantiated by incorporating a data constraint into a regular path template. A concrete query is then constructed by: (i) incorporating a start node into the query (ii) replacing label placeholders in path templates with concrete edge labels; and (iii) replacing attribute name placeholders in data constraint templates with concrete attribute names. Next we describe each component in detail.

Table 2 lists the 12 regular path templates used in the experiments. These are the top-12 most frequent property-path query patterns occurring in practice based on the study of publicly available query logs for SPARQL endpoints [3]. We use regular expressions in Table 2 as regular path templates in the experiments, and fix $k = 3$. Notice that edge labels here (e.g. a_1, a_2, etc.) are abstract placeholders which get instantiated with concrete labels.

Table 2: Regular templates used in the experiments.

Name	Type	Name	Type
Q_1	$(a_1\lvert\cdots\lvert a_k)^*$	Q_7	$a_1?/\cdots/a_k?$
Q_2	a^*	Q_8	$a/(b_1\lvert\cdots\lvert b_k)$
Q_3	$a_1/\cdots/a_k$	Q_9	$a_1/a_2?/\cdots/a_k?$
Q_4	a^*/b	Q_{10}	$(a/b^*)\lvert c$
Q_5	$a_1\lvert\cdots\lvert a_k$	Q_{11}	$a^*/b?$
Q_6	a^+	Q_{12}	$a/b/c^*$

We classify regular path templates into three categories based on their occurrence in query logs (see Table 5 in in [3]). The results of this classification are listed in Table 3.

Table 3: Categories of regular templates used in the experiments.

Category	Regular Templates	Relative Occurrence	Total Percentage
Frequently-used	Q_1, Q_2, Q_3, Q_4	>10%	87.58%
Occasionally-used	Q_5, Q_6, Q_7	>1% and <10%	12.34%
Rarely-used	$Q_8, Q_9, Q_{10}, Q_{11}, Q_{12}$	>0.01% and <1%	0.08%

Table 4 (column labelled "Description") lists the descriptions of 5 data constraint templates we use, including two *simple arithmetic constraints* D_1 and D_2, and three *complex arithmetic constraints* D_3, D_4 and D_5. Combining the five data constraint templates with the twelve regular path templates, gives us a total of 60 query templates. A concrete query is then constructed in two phases. First, we replace the label placeholders in path templates with concrete edge labels, and replace attribute name placeholders in data constraint templates with concrete attribute names, giving rise to a parametric regular expression `pregex`. We then select a starting node v for our parametric regular path query $(v, \texttt{pregex})$ (see Definition 3.7). An example of a partially instantiated query template (changing only

the edge labels) is given in Table 4 (rightmost column). The source vertices are chosen randomly from the graph, and replacing label placeholders with labels selected from most frequent edge labels occurring in the graph. These 100 concrete query instances per each template are used to evaluate the naive algorithm, the optimized algorithm, and we also use a version which removes the data constraints to compare the impact of incorporating data constraints into RPQs. The latter serves as a baseline which is supported by MillenniumDB, so it allows us to do a fair comparison.

How we ran the experiments? The evaluation is conducted on a Ubuntu `22.04` LTS subsystem on a Intel 13700H laptop assigned with 16 GB RAM. The experiments set a time-out threshold of 10 s for each query for all data sets except the Telecom dataset, and the time-out threshold for the Telecom dataset is set to 30 s due to its larger size and density.

Baseline. As a baseline we use the default implementation of RPQs in MillenniumDB. This means that for each query we generate, we have a baseline RPQ which simply removes all the data constraints. This will allow us to measure the effect of adding data constraints to RPQs.

Evaluation Metrics. We evaluate the performance of the algorithms based on three metrics: (1) running time of each query; (2) general memory consumption: each run was executed against a dedicated, freshly started instance of MillenniumDB, and the memory consumption was measured externally by monitoring the peak Resident Set Size (RSS) of MillenniumDB process during query execution; (3) oracle query memory consumption: we export the memory consumption of Z3 during a query evaluation by its `C` API; and (4) oracle query count: the number of invocations of the SMT solver recorded in the logs of MillenniumDB, which we treat as queries to an external oracle, because it is a new feature introduced to graph path queries.

6.2 Scalability Evaluation

This subsection studies the *feasibility* of the optimized algorithm (Algorithm 3). For this, we test how the optimized algorithm performs across different queries, graph sizes and graph density; i.e. we answer research questions **RQ1**, **RQ2** and **RQ3**. We start by measuring the time performance of our solution over different queries and different graph sizes.

Time Performance. We study the *running-time distribution* of the optimized algorithm. Although our macro-state algorithm increases significantly rather than normal regular path queries, the optimized algorithm still performs well across different datasets and queries.

Figure 5 presents the running time distribution across datasets and data constraints, and Figure 6 presents the running time distribution across datasets and regular templates. We observe that the optimized algorithm maintains a low median running time across datasets, and distributions over sparse graphs are more concentrated than those over dense graphs, while dense graphs, although running time distributions are wider, most queries can still be evaluated within 1 s. Such results indicate that *the optimized algorithm scales well with graph density and size*, and provide a positive answer to **RQ1** from the perspective of time performance.

The performance variation with different queries is also studied and we obtain positive results. According to the results in Figure 6, we observe that the optimized algorithm performs well over frequently-used templates with most queries can be finished within 1 s *no matter whether the data constraint is simple or complex*, which cover nearly 90% of real-world queries. According to the results in Figure 5, we observe that complex data constraints have better time performance than simple data constraints in terms of median running time and the concentrated distribution, while the simple data constraints have a longer tail in the distribution but most queries can still be evaluated within 1 s. These results indicate that *the optimized algorithm scales well with different queries*, and provide a positive answer to **RQ2** in terms of time performance.

Baseline comparison. While we showed that our optimized algorithm scales well over real-world graph, we still need to measure the impact of adding complex data comparisons to RPQs. For this, in Figure 7, we measure the time of running the regular pattern of each of tested queries and compare with the time of running parametric regular path queries. As can be observed, the optimized algorithm takes much more time compared to usual regular path queries by 10 times more average running time and wider distribution, as normal regular path queries can be evaluated within 15 ms in most cases in Figure 7. The reason for this is that the optimized algorithm does much more work since the data constraint portion of the query must be satisfied which requires an extensive amount of queries to the oracle during the evaluation. Next we measure the effect of oracle queries.

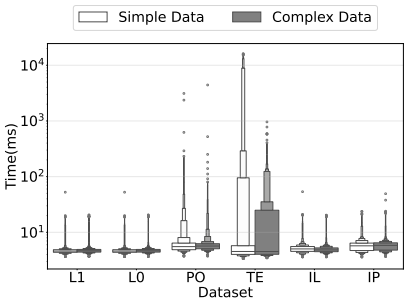


Figure 5: Running time distributions for queries across different datasets and data constraints in Table 4

Oracle Query Count. From the view of complexity analysis, the optimized algorithm invokes the oracle with a length c which is much smaller than the size of the graph $|V|+|E|$. As a result, the total count of oracle queries dominates the performance of a parametric regular path query. Statistical evidence in Figure 8 shows that the running time of an individual query is strongly correlated to the oracle query count during the evaluation by a correlation coefficient 0.964 with $p < 0.001$.

Figure 9 presents the oracle query count distributions across data constraints and datasets. We observe that the parametric regular queries in dense graphs produce much more oracle queries compared to sparse graphs, while most queries among the dense datasets keep oracle query count below 10^3, except the simple-data-constraint queries in TE. We also observe that the optimized

Table 4: Data constraints used in the experiment.

Name	Description	Example with a^*	Category
D1	The distance between the average of attribute *attr* and the values of *attr* should within a threshold c	$\{a_v, ?p - attr \leq c \wedge attr - ?p \leq c\}/$ $(\{a_e, \top\}/\{a_v, ?p - attr \leq c \wedge attr - ?p \leq c\})^*$	Simple
D2	Upper and lower bound of an attribute *attr*	$\{a_v, ?p \leq attr \wedge ?q \geq attr\}/$ $(\{a_e, \top\}/\{a_v, ?p \leq attr \wedge ?q \geq attr\})^*$	Simple
D3	The distance of upper and lower bound of attribute *attr* should be within threshold c.	$\{a_v, ?p \leq attr \wedge ?q \geq attr \wedge ?q - ?p \leq c\}/$ $(\{a_e, \top\}/\{a_v, ?p \leq attr \wedge ?q \geq attr \wedge ?q - ?p \leq c\})^*$	Complex
D4	Let the value of attribute $attr_1$ of the start point be a_{10}, and the value of attribute $attr_2$ of the start point be a_{20} for each successor along a path, $0.5 \cdot a_{10} + c_1 \leq attr_1$ and $\lvert a_{20} - attr_2 \rvert \leq c_2$	$\{a_v, ?p = attr_1 \wedge ?q = attr_2\}/$ $(\{a_e, \top\}/\{a_v, ?p \cdot 0.5 + c_1 \leq attr_1 \wedge ?q - attr_2 \leq c_2$ $\wedge attr_2 - ?q \leq c_2\})^*$	Complex
D5	Let the two-dimensional manhattan distance between the start point (x_1, y_1) and each node along a path (x_2, y_2) be within a threshold c.	$\{a_v, ?p = attr_1 \wedge ?q = attr_2\}/$ $(\{a_e, \top\}/\{a_v, ?p - attr_1 + ?q - attr_2 \leq c$ $\wedge attr_1 - ?q + ?p - attr_2 \leq c$ $\wedge attr_1 - ?q + attr_2 - ?p \leq c$ $\wedge ?q - attr_1 + attr_2 - ?p \leq c\})^*$	Complex

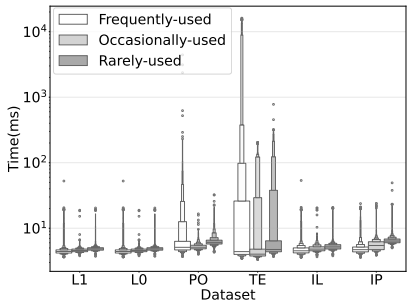


Figure 6: Running time distributions for queries across different datasets and regular templates in Table 3

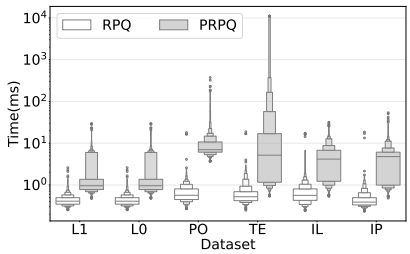


Figure 7: Running time of regular path queries and parametric regular path queries across datasets.

algorithm queries the oracle more times for *simple* data constraints, especially for large and dense graphs L1, TE and PO, which indicates that complex data constraints can terminate earlier, due to reaching a contradiction more easily.

According to Figure 10 and Figure 6, we observe that the queries with the total oracle query count below 10^3 can be evaluated within 100 ms in most cases, and for the extreme cases with oracle query

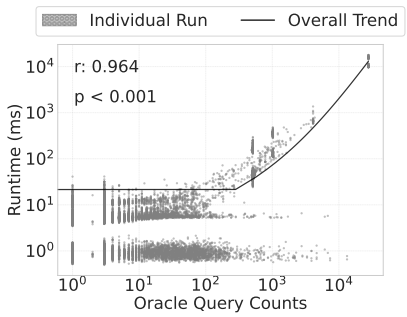


Figure 8: The distribution of each individual parametric regular query running time and its oracle query count

count above 10^4, the running time can also be within 10 s. With simple estimations, the average time for each oracle query is about the level of 1 ms, which is acceptable for most applications. These results indicate that the optimized algorithm scales well with *high oracle query counts*, and provide strong support for our complexity analysis and a positive answer to **RQ3**.

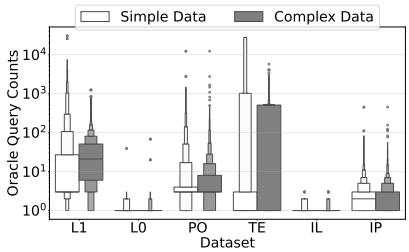


Figure 9: The distribution of oracle query counts across different data constraints in Table 4

Figure 10 presents the oracle query count distributions across regular templates and datasets. We observe that queries in frequently-used templates produce the most oracle queries among all templates, but most queries only produce oracle queries below 10^3, which indicates that 87.58% of real-world queries can be still evaluated efficiently. Both queries in occasionally-used and rarely-used templates produce less oracle queries, and the counts are moderate by most below 10^2.

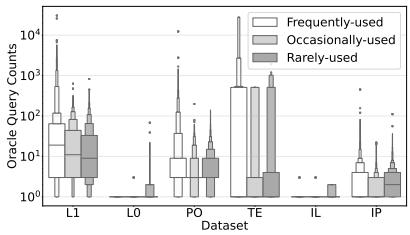


Figure 10: The distribution of oracle query counts across different regular templates in Table 3.

Memory Performance. The memory consumption of the optimized algorithm is moderate but increases markedly during regular path queries (Figure 11), which shows that the our macro-state approach is feasible in terms of memory consumption.

We attribute this increase to the cost of querying the oracle (the SMT solver), i.e. $M_{oracle} = M_{prpq} - M_{rpq}$, where M_{prpq} is the memory usage of a parametric regular path query, and M_{rpq} is the memory usage of a regular path query with the same start point and regular template as M_{prpq} respectively. We measure the memory consumption of the SMT solver M_{Z3}, and calculate the increasing memory $\Delta M = M_{prpq} - M_{rpq}$.

Table 5 shows a strong linear correlation between the sample-level differences of ΔM and Z3's memory usage for datasets L1, L0, PO, IL, and IP ($r = 0.963$–1.000, $p < 0.001$). The TE dataset was an exception, showing moderate correlations for D1 and D2 ($r = 0.454$–0.470, $p < 0.001$) but strong correlations for D3-D5 ($r = 0.869$–0.977, $p < 0.001$), indicating variability in early measures. The statistical analysis confirms that oracle queries are the principal cause of increased memory usage.

However, we have not observed a strong correlation between the memory consumption of oracle queries and graph size or density, or variation of queries and data constraints, which indicates that the memory consumption of oracle queries is mainly determined by the SMT solver itself rather than the graph or queries, which provides an answer to **RQ1**, **RQ2** and **RQ3** from the perspective of memory consumption.

Conclusions. Overall, even if the parametric regular path queries evaluation is NP-hard according to Theorem 3.13, the experimental results demonstrate the *feasibility* of the optimized algorithm (Algorithm 3) which scales well with large and dense datasets and complex data constraints.

6.3 Performance Comparison

This subsection foucuses on **RQ4** and compares the performance of the naive algorithm (Algorithm 1) and the optimized algorithm (Algorithm 3), showing a significant gain in deploying the optimizations described in Section 5.

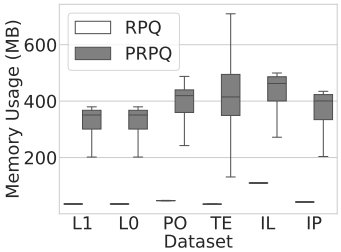


Figure 11: The distribution of parametric regular path queries and normal regular path queries memory consumption.

Table 5: Sample-level correlation between the memory usage differences ΔM and the memory consumption of Z3 with $p < 0.001$

Dataset	D1	D2	D3	D4	D5
L1	0.970	0.969	0.971	0.963	0.967
L0	0.999	0.999	0.999	0.999	0.999
PO	0.995	0.995	1.000	1.000	1.000
TE	0.454	0.470	0.869	0.977	0.946
IL	0.998	0.998	0.998	0.998	0.996
IP	0.998	0.998	0.998	0.998	0.995

Running Time Analysis. The optimized algorithm significantly improves the running time for queries that both algorithms can complete within the time limit. Figure 12 shows the running time distributions of both algorithms across all datasets and data constraints (for queries that do not time out). We observe that the optimized algorithm has lower median running time and more concentrated distribution compared to the naive algorithm for complex data constraints across the dense and large-size graphs. Although the performance of the naive algorithm is slightly better to that of the optimized algorithm on sparse graphs such as IL, the difference is marginal, and the general running time of optimized algorithm still remains fast.

Time-out Analysis. A significant gain of the optimized algorithm can be observed with respect to the number of queries that time out. That is, the optimized algorithm reduces the time-out rate considerably compared to the naive algorithm. Figure 13 presents the time-out rates of naive and optimized algorithm across data constraints and datasets where timeouts occur. We observe that *optimized algorithm improves the time-out markedly for complex data constraints*, where the naive algorithm fails to complete a markedly larger portion of queries within the time limit, while *most* queries are successfully evaluated by the optimized algorithm within the prescribed timeout threshold for each dataset.

Overall, these results provides positive evidence to **RQ4**, demonstrating that *the optimized algorithm significantly reduces the time-out rate and improves the running time, especially for complex data constraints.*

Figure 12: Running time across datasets and data constraints in Table 4

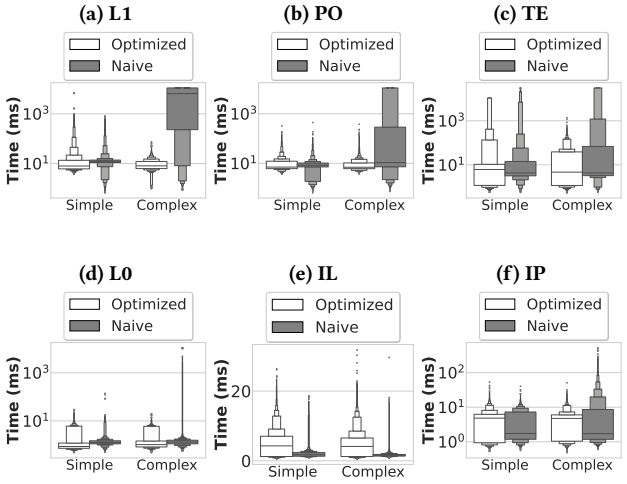
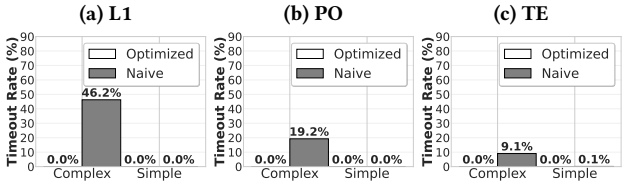


Figure 13: Time-out rate across datasets and data constraints in Table 4



7 CONCLUSIONS

In this paper we extend navigational query languages for graph databases with the constraints for reasoning on how data values change along the explored paths. To this end, we introduced parametric regular path queries and developed efficient algorithms for evaluation of these queries. We implemented our solution on top of MillenniumDB — an open-source industry-strength graph database system that fully supports regular path queries — showing its feasibility in real-world scenarios. Furthermore, our implementation of the algorithm constitutes the first scalable constraint database system, which can handle databases with up to tens of millions of edges. There are many avenues for future work, in particular supporting more intricate constraints. Although our algorithm can be extended to handle linear integer arithmetic, it does not immediately extend to *nonlinear* real arithmetic. In particular, *can we achieve this without the heavy machinery of constraint database theory?* We leave this as an open problem.

REFERENCES

[1] Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan L. Reutter, and Domagoj Vrgoč. 2017. Foundations of Modern Query Languages for Graph Databases. *ACM Comput. Surv.* 50, 5 (2017). https://doi.org/10.1145/3104031

[2] Pablo Barceló Baeza. 2013. Querying graph databases. In *Symposium on Principles of Database Systems (PODS)*. ACM, 175–188. https://doi.org/10.1145/2463664.2465216

[3] Angela Bonifati, Wim Martens, and Thomas Timm. 2017. An Analytical Study of Large SPARQL Query Logs. *Proc. VLDB Endow.* 11, 2 (2017), 149–161. https://doi.org/10.14778/3149193.3149196

[4] Aaron R. Bradley and Zohar Manna. 2007. *The Calculus of Computation - Decision Procedures with Applications to Verification.* Springer.

[5] Jo-Hag Byon and Peter Z. Revesz. 1995. DISCO: A Constraint Database System with Sets. In *Constraint Databases and Applications, ESPRIT WG CONTESSA Workshop, Friedrichshafen, Germany, September 8-9, 1995, Proceedings (Lecture Notes in Computer Science)*, Gabriel M. Kuper and Mark Wallace (Eds.), Vol. 1034. Springer, 68–83. https://doi.org/10.1007/3-540-60794-3_14

[6] Isabel F. Cruz, Alberto O. Mendelzon, and Peter T. Wood. 1987. A Graphical Query Language Supporting Recursion. In *International Conference on Management of Data (SIGMOD)*. 323–330. https://doi.org/10.1145/38713.38749

[7] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 337–340.

[8] Alin Deutsch, Nadime Francis, Alastair Green, Keith Hare, Bei Li, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Wim Martens, Jan Michels, Filip Murlak, Stefan Plantikow, Petra Selmer, Oskar van Rest, Hannes Voigt, Domagoj Vrgoč, Mingxi Wu, and Fred Zemke. 2022. Graph Pattern Matching in GQL and SQL/PGQ. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 2246–2258. https://doi.org/10.1145/3514221.3526057

[9] Alin Deutsch, Nadime Francis, Alastair Green, Keith W. Hare, Bei Li, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Wim Martens, Jan Michels, Filip Murlak, Stefan Plantikow, Petra Selmer, Oskar van Rest, Hannes Voigt, Domagoj Vrgoc, Mingxi Wu, and Fred Zemke. 2022. Graph Pattern Matching in GQL and SQL/PGQ. In *International Conference on Management of Data (SIGMOD)*. ACM, 2246–2258. https://doi.org/10.1145/3514221.3526057

[10] Benjamín Farias, Wim Martens, Carlos Rojas, and Domagoj Vrgoc. 2024. PathFinder: Returning Paths in Graph Queries. In *International Semantic Web Conference (ISWC)*. 135–154. https://doi.org/10.1007/978-3-031-77850-6_8

[11] Benjamín Farias, Carlos Rojas, and Domagoj Vrgoc. 2023. Evaluating Regular Path Queries in GQL and SQL/PGQ: How Far Can The Classical Algorithms Take Us? *CoRR* abs/2306.02194v1 (2023). https://doi.org/10.48550/ARXIV.2306.02194 arXiv:2306.02194

[12] Diego Figueira, Artur Jez, and Anthony W. Lin. 2022. Data Path Queries over Embedded Graph Databases. In *PODS*. ACM, 189–201.

[13] Diego Figueira, Anthony W. Lin, and Liat Peterfreund. 2025. Complexity of Evaluating GQL Queries. arXiv:2407.06766 [cs.DB] https://arxiv.org/abs/2407.06766

[14] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. 2018. Cypher: An Evolving Query Language for Property Graphs. In *International Conference on Management of Data (SIGMOD)*. ACM, 1433–1445. https://doi.org/10.1145/3183713.3190657

[15] Amélie Gheerbrant, Leonid Libkin, and Alexandra Rogova. 2025. Dangers of List Processing in Querying Property Graphs. *Proc. ACM Manag. Data* 3, 3 (2025), 144:1–144:25. https://doi.org/10.1145/3725281

[16] Stéphane Grumbach, Philippe Rigaux, Michel Scholl, and Luc Segoufin. 2000. The DEDALE Prototype. In *Constraint Databases*, Gabriel M. Kuper, Leonid Libkin, and Jan Paredaens (Eds.). Springer, 365–382.

[17] Steve Harris, Andy Seaborne, and Eric Prud'hommeaux. 2013. SPARQL 1.1 Query Language. W3C Recommendation. https://www.w3.org/TR/sparql11-query/

[18] ISO/IEC JTC 1/SC 32. 2024. *ISO/IEC 39075:2024. Information technology – Database languages GQL.* Technical Report. ISO.

[19] Artur Jez, Anthony W. Lin, Oliver Markgraf, and Philipp Rümmer. 2023. Decision Procedures for Sequence Theories. In *CAV (2) (Lecture Notes in Computer Science)*, Vol. 13965. Springer, 18–40.

[20] Guodong Jin, Xiyang Feng, Ziyi Chen, Chang Liu, and Semih Salihoglu. 2023. KÙZU Graph Database Management System. In *13th Conference on Innovative Data Systems Research, CIDR 2023, Amsterdam, The Netherlands, January 8-11, 2023*. www.cidrdb.org. https://www.cidrdb.org/cidr2023/papers/p48-jin.pdf

[21] Gabriel Kuper, Leonid Libkin, and Jan Paredaens. 2000. *Constraint Databases.* Springer.

[22] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. http://snap.stanford.edu/data.

[23] Leonid Libkin, Wim Martens, and Domagoj Vrgoc. 2016. Querying Graphs with Data. *J. ACM* 63, 2 (2016), 14:1–14:53. https://doi.org/10.1145/2850413

[24] Leonid Libkin, Cristina Sirangelo, and Deniz Yilmaz. 2025. Extending Pattern Matching Queries in Property Graphs with Interpreted Predicates. In *Proceedings of the 8th Joint Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA), Berlin, Germany, June 22-27, 2025*, Akhil Arora and Stefania Dumbrava (Eds.). ACM, 6:1–6:9. https://doi.org/10.1145/3735546.3735855

[25] Matteo Lissandrini, Martin Brugnara, and Yannis Velegrakis. 2018. Beyond Macrobenchmarks: Microbenchmark-based Graph Database Evaluation. *PVLDB* 12, 4 (Dec. 2018), 390—-403. https://doi.org/10.14778/3297753.3297759

[26] Wim Martens, Matthias Niewerth, Tina Popp, Carlos Rojas, Stijn Vansummeren, and Domagoj Vrgoc. 2023. Representing Paths in Graph Database Pattern Matching. *Proc. VLDB Endow.* 16, 7 (2023), 1790–1803. https://doi.org/10.14778/3587136.3587151

[27] International Consortium of Investigative Journalists. 2013. *ICIJ Offshore Leaks Database.* https://offshoreleaks.icij.org Accessed: 2025-08-18.

[28] International Consortium of Investigative Journalists. 2017. *Paradise Papers dataset.* https://www.icij.org/investigations/paradise-papers/ Accessed: 2025-08-18.

[29] Oracle. [n.d.]. Oracle Graph Database. https://www.oracle.com/database/graph/.

[30] Peter Z. Revesz, Rui Chen, Pradip Kanjamala, Yiming Li, Yuguo Liu, and Yonghui Wang. 2000. The MLPQ/GIS Constraint Database System. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, May 16-18, 2000, Dallas, Texas, USA*, Weidong Chen, Jeffrey F. Naughton, and Philip A. Bernstein (Eds.). ACM, 601. https://doi.org/10.1145/342009.336590

[31] Michael Sipser. 1997. *Introduction to the theory of computation.* PWS Publishing Company.

[32] L. Takac and M. Zábovský. 2012. Data analysis in public social networks. *International Scientific Conference and International Workshop Present Day Trends of Innovations* (Jan. 2012), 1–6.

[33] Jena Team. 2021. Jena TDB. https://jena.apache.org/documentation/tdb/

[34] Memgraph Team. 2023. Memgraph. https://memgraph.com/

[35] TigerGraph Team. 2021. TigerGraph Documentation – version 3.1. https://docs.tigergraph.com/

[36] Vesoft Inc/Nebula. 2023. NebulaGraph. https://www.nebula-graph.io/

[37] Domagoj Vrgoč, Carlos Rojas, Renzo Angles, Marcelo Arenas, Diego Arroyuelo, Carlos Buil-Aranda, Aidan Hogan, Gonzalo Navarro, Cristian Riveros, and Juan Romero. 2023. MillenniumDB: An Open-Source Graph Database System. *Data Intelligence* 5, 3 (08 2023), 560–610. https://doi.org/10.1162/dint_a_00229 arXiv:https://direct.mit.edu/dint/article-pdf/5/3/560/2158194/dint_a_00229.pdf

[38] Jim Webber. 2012. A programmatic introduction to Neo4j. In *SPLASH*. https://doi.org/10.1145/2384716.2384777

A PROOF OF THEOREM 3.12

Proof. We prove by structural induction on parametric regular expressions.

Base case: For atomic expression $E = (t, \phi)$, the parametric automaton of E is $Aut = (\mathsf{L}, \chi, Attr, Q, \{q_0\}, F, \Delta)$, where: (1) $\mathsf{L} = \{t\}$ (2) χ is all the global parameters in ϕ. (3) $Attr$ is all the properties in ϕ. (4) $Q = \{q_0, q_f\}$ (5) $\{q_0\}$ is the only start state. (6) $F = \{q_f\}$ is a set of final states. (7) $\Delta = \{(q_0, (t, \phi, false), q_f)\}$,

We can verify each $p \in [[(t, \phi)]]^{\mu}$ accepted by Aut according to Definition 3.4.

Induction Step. : For a concatenation expression E_1/E_2, if E_1 is translated to $Aut_1 = (\mathsf{L}_1, \chi_1, Attr_1, Q_1, \{q_{01}\}, F_1, \Delta_1)$, and E_2 is translated to $Aut_2 = (\mathsf{L}_2, \chi_2, Attr_2, Q_2, \{q_{02}\}, F_2, \Delta_2)$, we construct a parametric automaton $Aut' = (\Sigma, \chi, Attr, Q, \{q_01\}, F', \Delta')$, with $F = F_2$. we redirect all transition towards states in $F1$ to q_{02}. Formally

$$\Delta' = \Big(\Delta_1 \setminus \{(q, (t, \phi, inv), q_f), \mid q_f \in F_1\}\Big) \cup \Delta_2 \\ \cup \{(q, (t, \phi, inv), q_{02}) \mid \forall (q, (t, \phi, inv), q_1) \in \Delta_1, q_1 \in F_1\}$$

For each $p' \in [[E_1 \cdot E_2]]^{\mu}$, we have $p' = p_1 \cdot p_2$ and $\mu' = \mu_1 \cup \mu_2$ where $p_1 \in [[E_1]]^{\mu_1}$ and $p_2 \in [[E_2]]^{\mu_2}$. By induction hypothesis, p_1 is accepted by Aut_1 with μ_1, and p_2 is accepted by Aut_2 with μ_2. Then e has an accept run with the new transition in Δ' and the unified assignment μ'.

For an alternation expression $E_1 \mid E_2$, E_1 is translated to an automaton $Aut_1 = (\mathsf{L}_1, \chi_1, Attr_1, Q_1, \{q_{01}\}, F_1, \Delta_1)$, and E_2 is translated to an automaton $Aut_2 = (\mathsf{L}_2, \chi_2, Attr_2, Q_2, \{q_{02}\}, F_2, \Delta_2)$. We construct $Aut' = (\Sigma, \chi, Q, \Sigma, q'_0, F', \Delta')$ by introducing a new initial state q_0 and discarding q_{01} and q_{02}. The final states set is $F' = F_1 \cup F_2$, and let all transitions from q_{01} and q_{02} start from q'_0, formally

$$\begin{aligned} \Delta' = &(\Delta_1 \setminus \{(q_{01}, (t, \phi, inv), q) \mid q \in Q_1\}) \\ &\cup (\Delta_2 \setminus \{(q_{02}, (t, \phi, inv), q) \mid q \in Q_2\}) \\ &\cup \{(q'_0, (t, \phi, inv), q) \mid (q_{01}, (t, \phi, inv), q) \in \Delta_1\} \\ &\cup \{(q'_0, (t, \phi, inv), q) \mid (q_{02}, (t, \phi, inv), q) \in \Delta_2\} \end{aligned}$$

For each $p' \in [[E_1 \mid E_2]]^{\mu}$, we have $p' = p_1$ or $p' = p_2$ and $\mu' = \mu_1 \cup \mu_2$ where $p_1 \in [[E_1]]^{\mu}$ and $p_2 \in [[E_2]]^{\mu}$. By induction hypothesis, p_1 is accepted by Aut_1 with μ_1, and p_2 is accepted by Aut_2 with μ_2. Then p' has an accept run with the new transition from q'_0 with μ'.

For a Kleene star expression E^*, if E is translated to $Aut = (\mathsf{L}, \chi, Attr, Q, \{q_0\}, F, \Delta)$, the parametric automaton of E^* is $Aut' = (\mathsf{L}, \chi, Attr, Q', \{q_0\}, F', \Delta')$ with $Q' = (Q \setminus F)$ and $F' = \{q_0\}$, and we redirect all transitions in Δ_f to q_0, i.e.

$$\Delta' = \Big(\Delta \setminus \Delta_f\Big) \\ \cup \{(q, (t, \phi, inv), q_0) \mid (q, (t, \phi, inv), q_f) \in \Delta_f\}$$

For each $p' \in [[E^*]]^{\mu}$, we have $p' \in [[E^i]]^{\mu_i}$ where $i > 0$ or $p' = \epsilon$ with empty assignment. For the ϵ case, since the initial state is also the final state, then ϵ is accepted without any conditions. If we have $p' \in [[E^i]]^{\mu_i}$, by induction hypothesis, p' is accepted by Aut with μ_i, and then p has an accept run towards q_0 with assignment μ.

For an inverse expression $\hat{}E$, and E can be translated to $Aut = (\mathsf{L}, \chi, Attr, Q, \{q_0\}, F, \Delta)$, it is suffices to modify all transitions of Δ, such that $\Delta' = \{(q, (t, \phi, !inv), q') \mid (q', (t, \phi, inv), q) \in \Delta\}$. For each $p \in [[\hat{}E]]^{\mu}$, according to Definition 3.4, p satisfies the condition of inverse transitions.

Since ? and + are derivable from | and $*$, it is enough to show the above basic operators. □

B ADDITIONAL DETAILS OF ALGORITHMS

Algorithm 4 Path Construction

```
procedure GetPath(state = (v, q, e, prev, 𝓕))
    if prev = ⊥ then
        return v
    else
        return GetPath(prev).extend(e, v)
```

Algorithm 5 Check validity of next state with the SIMPLE semantics

```
function isValid(state, next, restrictor)
    s ← state
    if s.node = v and s.prev ≠ ⊥ then return ⊥
    while s ≠ ⊥ do
        if s.node == next.node and s.prev ≠ ⊥ then return ⊥
        s ← s.prev
    return ⊤
```

[need the condition of simple paths - **Heyang**]

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009